NAME: SHRUTI K
ROLL NO: 240701510

EXPERIMENT – 2

COMPARISON OF VISUAL LOW-CODE TOOLS WITH CONVENTIONAL SOFTWARE DEVELOPMENT

Aim

To understand how low-code platforms differ from hand-coded software development, considering their stages, resource requirements, strengths and constraints.

Introduction

There are different ways to build an application. One approach relies on developers writing the program logic themselves, while another uses visual platforms and configurable building blocks. This exercise outlines both approaches and considers their use in a hospital information system.

Traditional Development Approach

Traditional software development involves writing and maintaining the application's code using editors, compilers and integrated development environments. The development team is responsible for planning the database, implementing screens and services, defining business operations, preparing the deployment environment and carrying out tests. The result can be tailored closely to the requirements, although the process often takes more time and demands a skilled technical team.

Low-Code Development Approach

Low-code development uses a visual workspace to construct applications with forms, templates, reusable controls and configurable automation. Instead of coding every routine function, the developer arranges components and defines how they interact. This reduces repetitive programming and can make delivery faster. The trade-off is that some specialised features may be difficult to implement outside the platform's supported capabilities.

Example: Hospital Management System

The Hospital Management System provides a practical example. Using Zoho Creator, a developer can configure patient and doctor records, appointments, admissions and billing through visual forms and workflows. Building the same system conventionally would involve coding its data storage, user interface, service layer, business rules and deployment process.

Traditional Development Lifecycle

A conventional project follows a sequence of engineering activities: analyse requirements, design the architecture, implement the software, perform quality checks, integrate modules, deploy the system and provide ongoing support.

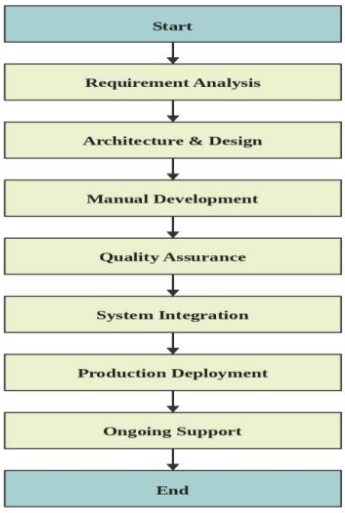


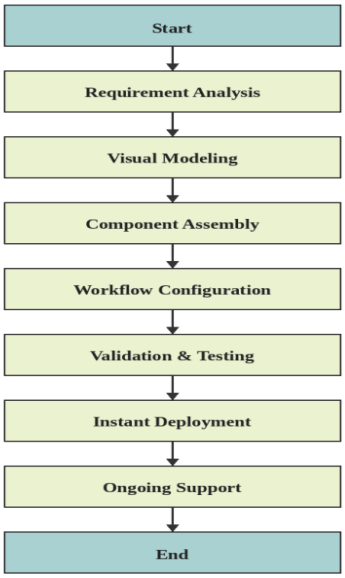
Figure 1: Traditional development lifecycle

[redacted]

Figure 1: Stages in the traditional software development lifecycle.

Low-Code Development Lifecycle

In the low-code process, requirements are first identified. Visual models and platform components are used to create the application, after which workflows are configured. Testing and validation take place before



deployment, followed by platform-based maintenance.

Figure 2: Stages in a low-code development process using Zoho Creator.

Comparative Analysis

Feature	Traditional Development	Low-Code Development
Development Time	More time is generally needed to code, integrate and test each feature.	Reusable elements and visual configuration can shorten the build cycle.
Required Expertise	Programmers need a strong understanding of languages, frameworks and system design.	Basic solutions can be assembled by users with little programming knowledge.
Customisation	Developers can implement a wide range of application-specific functions.	The extent of customisation is governed by platform features and integration support.
Expense	Initial investment can be substantial due to skilled labour and infrastructure.	Entry costs may be lower, but ongoing platform charges should be considered.
Ongoing Support	The team handles source-code modifications, bug fixes and regression testing.	The provider maintains the platform, while application workflows still need user attention.
Platform Lock-in	Source-code ownership gives greater freedom to change hosting or tools.	Moving away from the platform may require migration or redevelopment.
Best Fit	Large systems with complex rules, strict controls or high performance needs.	Quick prototypes, departmental applications and repetitive business processes.

Conclusion

Both development methods have useful roles. Conventional coding is preferable where control, extensibility and specialised engineering are essential. Low-code tools are effective for rapidly creating common forms and workflows. A hospital application can therefore use Zoho Creator for standard data-entry and process automation, while more complex services can be developed with traditional programming.